OceanBase × LangChain Community Course Series

Building Agentic RAG with LangChain & OceanBase

Episode 3 Corrective RAG with LangChain

Haili Zhang · LangChain Ambassador


OCEANBASE

×


LangChain Community

Haili Zhang

LangChain Ambassador

- GitHub @webup
- Twitter @zhanghaili0610
- Bilibili 沧海九粟

What We'll Cover

Master Corrective RAG patterns with LangChain v1 Middleware

Self-Correcting RAG

Self-Correction

Validate & recover from bad retrievals

Concept

Key Components

Grade + Fallback

Document grading & web search

Architecture

Implementation

2 Styles

Wrap-Style vs Node-Style middleware

Code

Self-Correcting RAG

RAG Knows When It's Wrong

Self-correction through validation and intelligent fallback

The Problem with Basic RAG

Why retrieval alone isn't enough

Retrieval Failures

- **Query-Document Mismatch**
  User asks about Q3 2024, retrieves Q2 data
- **Outdated Information**
  Knowledge base lacks recent data
- **Semantic Gap**
  Similar words but different meaning

Impact on Users

- **Hallucinated Answers**
  LLM generates plausible but wrong info
- **Loss of Trust**
  Users lose confidence in the system
- **No Recovery Path**
  System can't detect or fix errors

Key Insight: Basic RAG blindly trusts retrieval results

CRAG: Self-Correcting RAG

Adding intelligence to retrieval validation

Definition

Corrective RAG (CRAG) adds a self-correction layer that validates retrieval quality and falls back to alternative sources when documents are irrelevant.

Validation

Grade retrieved docs for relevance

- Relevance scoring (binary, 0-1, 0-10)
- LLM-based evaluation

Fallback

Alternative sources when KB fails

- Web search (Tavily, SerpAPI, etc.)
- Real-time information

Agentic

Agent decides next action

- Multi-step reasoning
- Retry with different queries

CRAG = Agentic RAG + Self-Correction Loop

CRAG Architecture

The complete corrective flow

> **Note**
>
> This is Agentic RAG – the agent may invoke the RAG tool multiple times with different queries before reaching a final answer.

arXiv

Corrective Retrieval Augmented Generation

Yan et al. 2024 – Introduces lightweight retrieval evaluator to assess document quality and trigger web search for low-confidence retrievals

RAG Tool: OceanBase Vector Store

Building the retriever tool that CRAG will grade and correct

```python
from langchain_oceanbase.vectorstores import OceanbaseVectorSt

# 1. Setup OceanBase vector store
vector_store = OceanbaseVectorStore(
    embedding_function=embeddings,
    table_name="langchain_knowledge_base",
    connection_args=connection_args,
    vidx_metric_type="cosine",
)

# 2. Create retriever (returns top-k doc
retriever = vector_store.as_retriever(search_kwargs={"k": 3})

# 3. Wrap as tool with @tool decorator
@tool
def search_nike_report(query: str) -> str:
    """Search Nike's 10-K annual report """
    results = retriever.invoke(query)
```

Implementation Details

OceanBase Vector Store

- ✓ Distributed vector storage
- ✓ Cosine similarity search
- ✓ Metadata filtering support

@tool Decorator

- ✓ Docstring = tool description for LLM
- ✓ Type hints for parameter schema
- ✓ Returns formatted string results

Why Wrap Retriever as Tool?

Agent Control

Agent decides when/how to search via ReAct loop

Middleware Access

wrap_tool_call can intercept and grade results

Tip: Tool combines multiple docs into a single formatted string - LLMs process text better than complex objects, and grading middleware can easily evaluate the concatenated content.

⭐ seekdb by OceanBase

AI-native search database - the recommended backend for this course

Unified Multi-Model Engine

Relational | Vector | Full-Text | JSON | GIS

AI Framework Integrations

LangChain | LlamaIndex | Dify | MCP Server | MySQL Driver

Why seekdb for RAG?

- ✓ Hybrid search (vector + full-text) in single query
- ✓ ACID transactions for data integrity
- ✓ MySQL compatible - easy migration

✓ Same langchain-oceanbase code works - just update connection string

seekdb

AI-Native Search Database

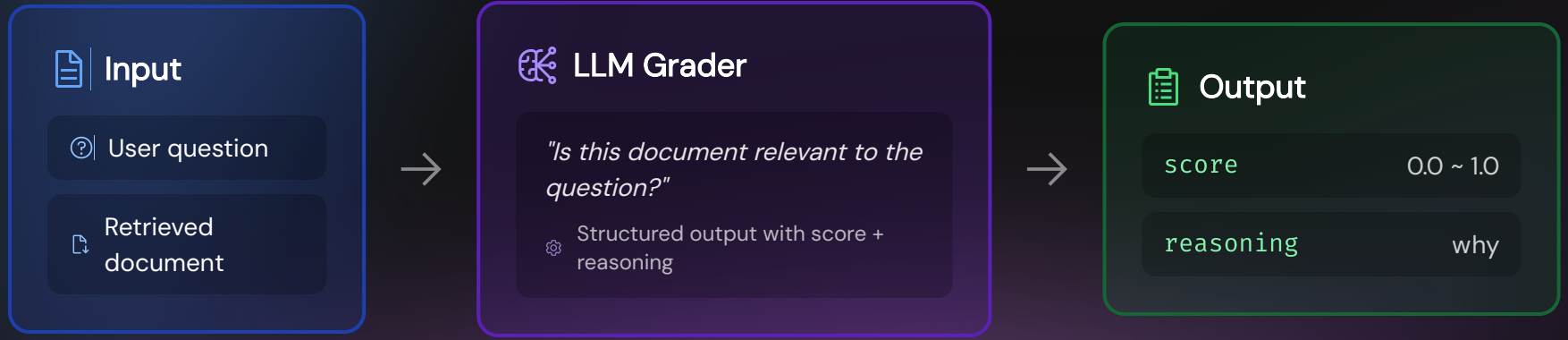
by OceanBase

oceanbase.ai →

Try seekdb Free

Document Grading

LLM-based relevance scoring with structured output



Structured output may fail with some models - always add error handling

Fallback Strategy

When and how to recover from irrelevant retrievals

When to Trigger

Condition check before each model call

```
if (
  documents_relevant == False
  and used_fallback == False
  and last_query exists
):
  trigger_fallback()
```

- ✓ Prevents infinite loops
- ✓ Ensures fallback runs only once

Fallback Options

Web Search — Primary

Tavily API for real-time comprehensive results

Alternative KB

Different vector store or broader search scope

Query Rewrite

Reformulate query and retry RAG tool

This notebook uses Tavily web search as the primary fallback strategy

Implementation

</> Agent + Middleware = CRAG

Building self-correcting RAG with LangChain v1

How LangChain Enables CRAG

Combining React Agent with Middleware for self-correction

React Agent

The foundation for agentic RAG

- ReAct loop (Reason + Act)
- Tool calling capability
- Multi-step reasoning

Middleware Hooks

Add reflection without changing agent code

- Observe tool outputs
- Modify model behavior
- Track custom state

Middleware = Clean separation between agent logic and CRAG validation

Agent Loop with Middleware

Six hooks intercept every step of agent execution

Middleware Hooks

Four hooks to intercept agent behavior

`wrap_tool_call`

Intercept and process tool outputs

→ CRAG: Grade retrieved documents

`before_model`

Inject state or trigger fallback

→ CRAG: Trigger web search fallback

`wrap_model_call`

Modify tools available to model

→ CRAG: Filter tools by state

`after_model`

Log or post-process responses

→ CRAG: Debug & trace

How hooks implement CRAG flow:

RAG Tool → `wrap_tool_call` (Grade) → `before_model` (Fallback?) → Answer

Custom State Schema

Track grading results and control flow

State vs Context

`State` **Mutable during run**

TypedDict extending AgentState

`Context` **Immutable dependencies**

Static data like user ID

CRAG State Fields

`messages`	`from AgentState`
`last_rag_query`	`for fallback`
`documents_relevant`	`bool | None`
`used_web_fallback`	`bool`

State updates are returned as dict from before_model / after_model hooks

Grading: wrap_tool_call

Intercept RAG tool output and grade document relevance

```python
def wrap_tool_call(self, request: ToolCallRequest,
    handler: Callable[[ToolCallRequest], ToolMessage],
) → ToolMessage:
    result = handler(request)  # Execute tool first

    if request.tool_call.get("name") == self.rag_tool_name:
        query = request.tool_call.get("args", {}).get("query")
        grade = self.grader.invoke({
            "question": query, "document": result.content[:200

        is_relevant = grade.binary_score.lower() == "yes"
        self._pending_grading = {  # Store for before_model
            "last_rag_query": query,
            "documents_relevant": is_relevant,
            "grade_reasoning": grade.reasoning
        }
    return result  # Must return ToolMessage, not state
```

Grading Flow

Two-Phase Pattern

1. wrap_tool_call: grade & store
2. before_model: apply to state

Why? wrap_tool_call MUST return ToolMessage

Structured Output

```python
class GradeResult:
  binary_score: str # "yes"/"no"
  reasoning: str

# with_structured_output()
```

Design Principles

Focus on Relevance

Ask "does this help answer?" not "is this correct?"

Require Reasoning

Forces LLM to explain, reduces hallucination

Handle Errors

Some models fail structured output - add try/except

Tip: Custom state is defined via `state_schema = CorrectiveRAGState` in middleware class - middleware owns and manages the extended state fields.

Two Fallback Styles

Choose your control vs flexibility trade-off

Documents Irrelevant
score < threshold

→

Need Alternative
KB doesn't have answer

→

Web Search Fallback
Real-time information

Node-Style

Who decides?	Middleware
Implementation	`before_model injects`
Control level	Deterministic
Best for	Strict policies

Wrap-Style

Who decides?	Model
Implementation	`wrap_model_call filters`
Control level	Flexible
Best for	Agentic behavior

Node-Style: before_model

Middleware directly injects fallback result

```
                runtime: Runtime) → dict[str, Any] | None:
    """When RAG failed, perform web search."""
    documents_relevant = state.get("documents_relevant")
    used_web_fallback = state.get("used_web_fallback", False)
    last_rag_query = state.get("last_rag_query")

    should_search = (documents_relevant is False
                     and not used_web_fallback
                     and last_rag_query)

    if should_search:
        web_result = self.web_search_tool.invoke(last_rag_que
        injection_msg = AIMessage(content=f"Web search [web_result]

        return {
            "used_web_fallback": True,
            "messages": [injection_msg],
        }
```

How It Works

Fallback Conditions

- documents_relevant is False
- used_web_fallback == False
- last_rag_query exists

AIMessage Injection

Middleware calls tool & injects result as AIMessage

- → Model sees web results in context
- → Summarizes for user naturally

Why "Node-Style"?

Deterministic

Middleware decides fallback, not the model. Guaranteed execution when conditions met.

Like a Graph Node

Acts like a conditional node in LangGraph - checks state, executes action, returns update.

Enhancement: Use LLM to rewrite query before search - convert RAG query to web-optimized search terms for better results.

Wrap-Style: wrap_model_call

Model decides from filtered tool options

```python
def wrap_model_call(
    self, request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelCallResult:
    """Filter tools based on current state."""
    documents_relevant = request.state.get("documents_relevant")
    used_web_fallback = request.state.get("used_web_fallback")
    should_enable_web = (
        documents_relevant is False and not used_web_fallback
    )

    if should_enable_web:
        filtered_tools = request.tools  # All tools
    else:
        filtered_tools = [t for t in request.tools
                          if t.name == search_knowledge_base]

    modified_request = request.override(tools=filtered_tools)
    return handler(modified_request)
```

How It Works

Tool Filtering Logic

```
docs_relevant=False:
  → Enable all tools
docs_relevant=True:
  → RAG tool only
```

Start restricted, unlock web search after RAG fails

Handler Pattern

Wrap the model call chain:

```
1 request.override(tools= ... )
2 handler(modified_request)
```

Modified request flows through

Why "Wrap-Style"?

Flexible & Agentic

Model retains agency - decides IF and HOW to call tools. Natural ReAct behavior.

Constraint

Can only filter from tools registered with `create_agent(tools=[ ... ])`

Assembling the CRAG Agent

Putting all the pieces together

Wrap-Style (Tool Filtering)

```
wrap_agent = create_agent(
    tools=[rag_search, web_search],  # Both
    model=chat_model,
    middleware=[
        grading_middleware,
        tool_filtering_middleware,
    ],
    state_schema=CorrectiveRAGState,
)
```

- Both tools registered, filtered by state
- Model decides which to call

Node-Style (Manual Fallback)

```
node_agent = create_agent(
    tools=[rag_search],  # RAG only
    model=chat_model,
    middleware=[
        grading_middleware,
        fallback_middleware,
    ],
    state_schema=CorrectiveRAGState,
)
```

- Only RAG tool registered
- Middleware calls web search directly

System Prompt Tip: Keep it generic - don't mention specific tool names. This prevents model from "hallucinating" tool calls for filtered tools.

Try It Yourself

Complete implementation in Jupyter notebook

What's in the notebook:

- Full CRAG implementation
- Both middleware styles
- OceanBase vector store
- Tavily web search fallback

Key Highlights

- 🔍 Document grading with structured output
- 🔄 Two fallback styles comparison
- 🛠️ Complete middleware implementation

04 - Corrective RAG with LangChain v1 Middleware

🎯 Learning Objectives

By the end of this notebook, you will:

- Understand Corrective RAG (CRAG) architecture with self-correction
- Master LangChain v1's middleware system (`wrap_tool_call`, `before_model`, `wrap_model_call`)
- Implement document grading as tool call middleware
- Add web search fallback when retrieved documents are irrelevant
- Compare two middleware styles: Wrap-Style vs Node-Style

⚖️ What is Corrective RAG?

What We Learned

Key Takeaways

Core concepts and patterns to remember

Key Takeaways

What you should remember from this episode

CRAG = Self-Correcting RAG

Add validation and fallback to handle retrieval failures gracefully

- ✓ Grade documents for relevance before generating answers
- ✓ Fall back to web search when knowledge base fails

Why React Agent Works

LangChain React Agent provides the perfect foundation for CRAG

- ReAct loop naturally supports multi-step correction
- Middleware hooks enable reflection without changing agent code

LangChain Middleware Pattern

Intercept agent behavior without modifying core logic

`wrap_tool_call`	Grade after tool execution
`before_model`	Inject fallback or update state
`wrap_model_call`	Filter available tools

Two Implementation Styles

Choose based on your control vs flexibility needs

Node-Style
Deterministic, middleware calls tools directly

Wrap-Style
Flexible, model chooses from filtered tools

Next: Try the notebook and build your own CRAG agent!

Thank You ❤️

Build production-ready RAG systems with LangChain & OceanBase

LangChain

OceanBase

seekdb